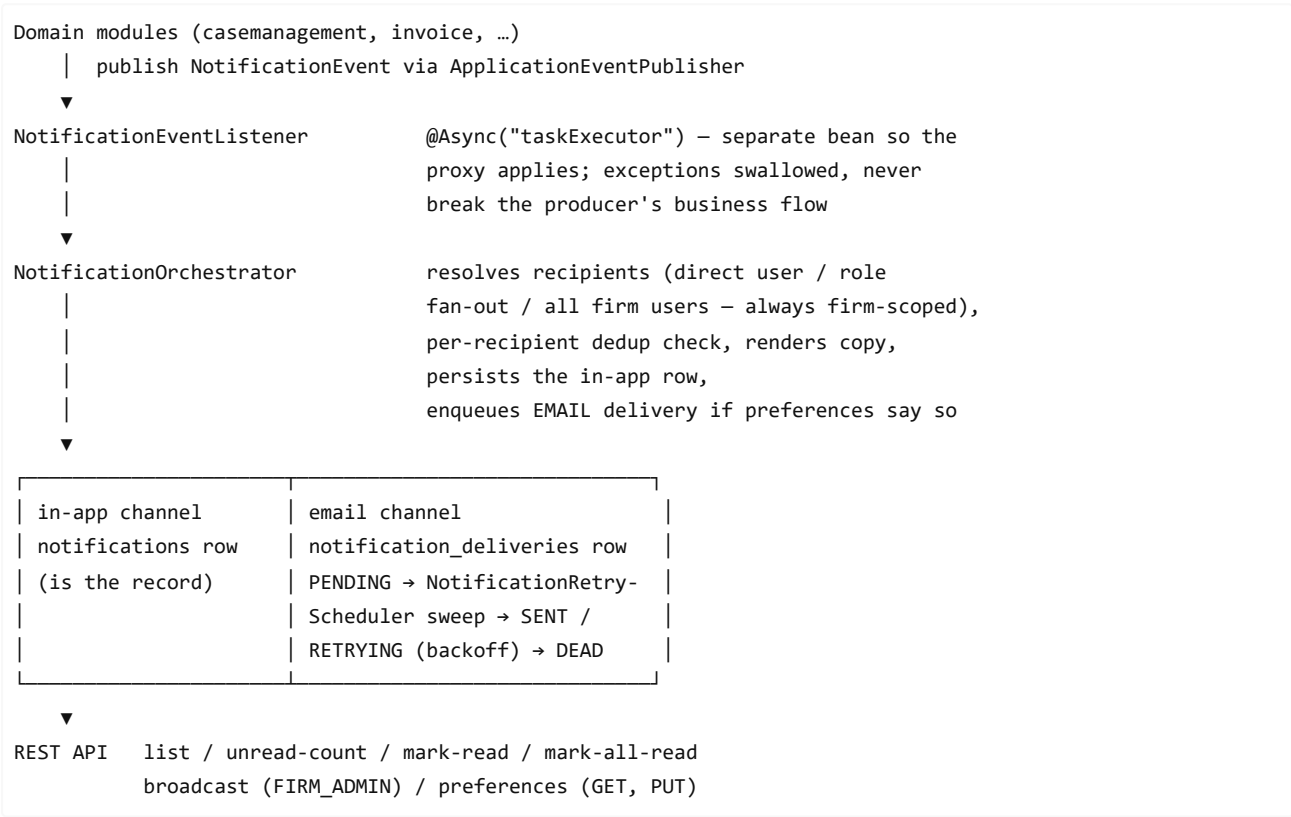


Notification Module

In-app notifications for the portal — the bell icon: a per-user feed, an unread badge, read-state management, an email follow-up channel with retry, firm-admin broadcasts, and per-user preferences. Driven by an in-process event pipeline that domain modules publish into.

1. Overview & Architecture



Scope decisions (with the trigger that would change them):

- **Polling only** — frontend polls `unread-count` every 45–60s. [SSE push only if users demand real-time]
- **Code-level rendering** — `NotificationRenderer`, exhaustive switch over types. [DB templates when Nepali/English i18n lands]
- **ALERT email opt-out locked** — hearing/appeal alerts always email; a user must not silently miss a deadline.
- Taxonomy **SYSTEM | ALERT | BROADCAST**; `TRANSACTIONAL` (password reset, MFA) intentionally excluded — owned by `EmailService` as request-response flows.

Swapping the in-process bus for a broker later means replacing `NotificationEventListener` — producers never change.

2. Data Model

notifications — the in-app channel

Column	Type	Notes
<code>id / uuid</code>	UUID	PK + secondary id (AuditableEntity)
<code>firm_id</code>	UUID NOT NULL	tenant scope — enforced in every query path
<code>recipient_user_id</code>	UUID NOT NULL	one row per recipient (fan-out at write time)
<code>type</code>	VARCHAR(40) NOT NULL	enum <code>NotificationType</code>

category	VARCHAR(20) NOT NULL	SYSTEM/ALERT/BROADCAST (pinned from type)
title, body	NOT NULL	rendered copy
reference_type, reference_id	nullable	polymorphic link: MATTER, INVOICE, COURT_EVENT, COURT_CASE, FIRM
read_at	nullable	NULL = unread
dedup_key	nullable	idempotency key, unique per recipient

Indexes: (recipient_user_id, read_at) hot path, (firm_id) , unique (dedup_key, recipient_user_id) .

notification_deliveries — out-of-band dispatch log

Column	Notes
notification_id → FK, recipient_user_id, firm_id	what/who
channel	EMAIL (SMS/PUSH reserved)
status	PENDING → SENT, or FAILED → RETRYING → ... → DEAD
attempts, last_attempted_at, next_attempt_at	retry bookkeeping
delivered_at, error_message	outcome

Index (status, next_attempt_at) serves the sweep.

notification_preferences

(user_id, type) unique — email_enabled boolean. Absent row = category default (ALERT → on, others → off).

Enums

- NotificationCategory : SYSTEM , ALERT , BROADCAST
- NotificationType : CASE_ASSIGNED , INVOICE_STATUS , APPEAL_LAPSED (SYSTEM) · HEARING_REMINDER , APPEAL_DEADLINE (ALERT) · ANNOUNCEMENT (BROADCAST)
- DeliveryChannel : EMAIL (SMS, PUSH reserved) · DeliveryStatus : PENDING/SENT/FAILED/RETRYING/DEAD

3. API Reference

Base: /api/v1/notifications — user-scoped endpoints resolve identity from the JWT.

Method	Path	Auth	Purpose
GET	/notifications?unreadOnly=&page=&size=	any authenticated	paged feed, newest first, size clamped 1–100
GET	/notifications/unread-count	any authenticated	badge endpoint (poll 45–60s) → {count}
POST	/notifications/{id}/read	any authenticated	stamp read_at (idempotent); 404 missing, 403 foreign
POST	/notifications/read-all	any authenticated	bulk read → {updated}
POST	/notifications/broadcast	hasRole('FIRM_ADMIN')	send ANNOUNCEMENT
GET	/notifications/notifications — see below	any authenticated	preferences view
PUT	/notifications/preferences	any authenticated	update one preference

Broadcast — `POST /api/v1/notifications/broadcast` :

```
{ "data": { "title": "Firm closed on dashain", "body": "Back on the 16th.", "audience": "ALL" } }
```

`audience` = `ALL` (every user in the firm) or a role code (`ADVOCATE` , `PARALEGAL` , `CLIENT` , `FIRM_ADMIN`). A role audience with **no users in the firm is rejected** (`BusinessRuleException`) — typos fail loudly instead of silently no-opping. Processes synchronously so the admin sees results immediately; event always carries the caller's `firmId` .

Preferences — `GET /api/v1/notifications/preferences` :

```
{ "data": [ { "type": "HEARING_REMINDER", "emailEnabled": true, "locked": true },
             { "type": "CASE_ASSIGNED", "emailEnabled": false, "locked": false } ] }
```

`PUT /api/v1/notifications/preferences` with `{ "data": { "type": "CASE_ASSIGNED", "emailEnabled": true } }` . Opting **out** of an ALERT type returns `BusinessRuleException` — deadlines must not be silencable.

Envelope is the platform standard `{success, message, responseCode, data}` .

4. Producing Notifications (for backend devs)

```
// Direct recipient (SYSTEM)
eventPublisher.publishEvent(NotificationEvent.toUser(
    firmId, assigneeUserId, NotificationType.CASE_ASSIGNED,
    "MATTER", matter.getId(),
    Map.of("matterNumber", matterNumber, "assignmentRole", role.name())));

// Role fan-out within the firm
eventPublisher.publishEvent(NotificationEvent.toRole(
    firmId, RoleCode.FIRM_ADMIN, NotificationType.INVOICE_STATUS,
    "INVOICE", invoice.getId(),
    Map.of("invoiceNumber", invoiceNumber, "status", newStatus.name())));

// Scheduled/alert events use the full constructor to set a day-bucketed dedupKey:
new NotificationEvent(firmId, recipientId, null, false,
    NotificationType.HEARING_REMINDER, "COURT_EVENT", eventId,
    "HEARING_REMINDER:COURT_EVENT:" + eventId + ":" + recipientId + ":" + date,
    Map.of("matterNumber", ..., "courtName", ..., "hearingTime", ...));
```

Rules:

- Exactly one targeting mode per event (`recipientUserId` | `recipientRoleCode` | `allFirmUsers=true`) — otherwise the orchestrator throws.
- Set dedupKey for anything a scheduler can re-run.** Convention: `TYPE:REFERENCE:refId:userId[:bucket]` . Downstream, the unique index (`dedup_key` , `recipient_user_id`) makes re-runs no-ops.
- Adding a new type: extend `NotificationType` (pin its category), add a renderer branch (the exhaustive switch makes a missing branch a compile error), add copy constants, write the producer test.

Live producers:

Producer	Event	Recipients	dedupKey
<code>CaseAssignmentServiceImpl.assign</code>	<code>CASE_ASSIGNED</code>	assigned user	—
<code>InvoiceServiceImpl.updateStatus</code>	<code>INVOICE_STATUS</code>	<code>FIRM_ADMIN</code> fan-out	—

HearingReminderServiceImpl (T-1, 08:00)	HEARING_REMINDER	attending advocate + linked our-clients	day-bucketed
AppealDeadlineEngine.checkUpcomingDeadlines (T-3, 03:05)	APPEAL_DEADLINE	case advocate (or FIRM_ADMIN if none)	bucketed by deadline date
AppealDeadlineEngine.closeLapsedAppeals (03:00)	APPEAL_LAPSED	FIRM_ADMIN fan-out	—
BroadcastService (admin compose)	ANNOUNCEMENT	ALL or role	—

5. Email Channel & Retry Sweep

- The orchestrator consults `NotificationPreferenceService.isEmailEnabledFor(userId, type)` per recipient; if enabled it persists a `PENDING` `notification_deliveries` row due immediately.
- `NotificationRetryScheduler` runs every minute (`fixedDelay=60s`): picks up due `PENDING/RETRYING` rows, resolves the parent notification, and hands them to the channel's `NotificationDispatcher` (strategy map — EMAIL today; SMS/PUSH plug in as new implementations).
- Outcomes: `SENT` is terminal; `FAILED` schedules exponential backoff **1m → 5m → 30m** with `attempts` capped at 3, then **DEAD**. A row with no registered dispatcher (or a vanished parent notification) goes `DEAD` rather than looping forever. One row's failure never stops the sweep.
- Emails render through `email/notification.html` (Thymeleaf, brand-aware like every platform email) and are sent by `EmailServiceImpl.sendNotificationEmail` — same SMTP resolution chain (firm SMTP → global DB config → default sender), same audit logging (`EMAIL_SENT` / `EMAIL_FAILED`).
- The dispatcher never throws; failures land on the delivery row as data, not exceptions.

6. Security & Multi-Tenancy

- Every endpoint resolves the user from the JWT (`CurrentUserResolver`); unauthenticated → `401` .
- Mark-read checks ownership: `recipient_user_id` mismatch → `403` .
- Role fan-out, broadcast audiences, and preference access are all firm-scoped or caller-scoped by construction.
- Broadcast is `@PreAuthorize("hasRole('FIRM_ADMIN')")` at the controller — consistent with the other firm-admin controllers.
- No new security-config entries: `/api/v1/notifications/**` falls under `anyRequest().authenticated()` .

7. Frontend Integration Guide

- Badge** — poll `GET /unread-count` every 45–60s; refresh after any mark-read call.
- Panel** — on bell click `GET /notifications?page=0&size=20` ; paginate via `page / totalPages` ; `unreadOnly=true` for an unread tab.
- Click-through** — map `referenceType` → route client-side (`MATTER` → `/matters/{id}` , `INVOICE` → `/invoices/{id}` , `COURT_EVENT` / `COURT_CASE` → the case's calendar/detail, `FIRM` → no navigation for announcements). On click: `POST /{id}/read` , then navigate.
- Category styling** — `SYSTEM` (info), `ALERT` (warning), `BROADCAST` (announcement).
- Compose UI** (firm admin only) — title + body + audience selector (`ALL` or role chips); surface the "no users in role" validation error.
- Settings page** — render `GET /preferences` ; disable toggles where `locked: true` with a "required alerts" hint; `PUT` on change.

8. Testing

Unit tests (Mockito, service layer — house style):

<code>NotificationRendererTest</code>	copy per type (all 6 types)
<code>NotificationEventTest</code>	factories, immutability, category pinning
<code>NotificationEventListenerTest</code>	delegation + exception swallowing
<code>NotificationOrchestratorTest</code>	targeting, fan-out, dedup, no-targeting rejection

NotificationOrchestratorEmailTest	email enqueue rules driven by preferences
NotificationServiceImplTest	scoping, authz on mark-read, mark-all
NotificationPreferenceServiceImplTest	defaults, ALERT lock, upsert, auth
NotificationRetrySchedulerTest	SENT terminal, backoff, DEAD cap, isolation
EmailNotificationDispatcherTest	success/failure/missing-user/throw-swallow
NotificationControllerTest	web layer, envelope, 403 advice
BroadcastServiceTest	ALL/role/invalid-audience/scoping
CaseAssignmentNotificationTest	producer publish + no-publish on rejection
InvoiceStatusNotificationTest	producer publish + no-publish on illegal transition
HearingNotificationProducerTest	T-1 alert events + dedup key + emails unchanged
AppealDeadlineNotificationTest	T-3 alert + stable dedup + lapsed notice

Run:

```
./mvnw test # 337 tests green at ship time
```

Manual smoke: assign a user to a matter → their bell count increments within a poll cycle; tomorrow's hearings (PESHI/SCHEDULED) → in-app alert + email to advocate/clients; a DECIDED case with deadline in ≤3 days → T-3 alert; firm admin broadcast → all/role users get it; opt out of CASE_ASSIGNED email → no delivery row on next event.

9. Follow-ups (deliberately not built)

- **SSE/WebSocket push** — only if real-time demand materializes; polling is the approved v1 UX.
- **SMS / PUSH channels** — new `NotificationDispatcher` implementations; sweep and delivery rows already support them.
- **Digest option** — daily summary email (v2 nice-to-have).
- **DB-backed templates** — when Nepali/English i18n lands; renderer branches map 1:1 to template rows.
- **Broadcast to role+firm-admin history/audit UI** — fan-out works; an admin "sent announcements" list is pure CRUD on existing rows.